Sections 01–11 are dated: August 2026. The explanations still hold; the numbers are from the August models. Current (22 Oct 2026): violence 88.0% of 200 held-out fights at 1.4 false alarms/hr on 5 h of cameras (two models averaged, every alarm confirmed against crops around the people present), 93.1% accuracy on 1,310 test clips; weapons v3 89.6% / 39.4% recall on large / street-size guns; vandalism re-enabled (experimental), 87.1% at 0.6/hr; robbery (experimental) 0.5/hr on its own. **With the Qwen3-VL-2B alarm double-check** (in the app, experimental): fights 94.0% at 0.4/hr, robbery 59.4% of events at 0.6/hr, false ARMED THREAT alerts 2.8 → 1.4/hr. See `docs/detection_performance_report.md` §1. Section 00 below is current.

DEFENSE REFERENCE · ECOVISION SENTINEL

Three detectors, measured

What each model actually does, how it behaves on real footage, and exactly where each one stops working. Every figure here comes from a script in the repository — where a number rests on something thin, it says so on the same line.

GTX 1660 SUPER · written 2026-08-14 · section 00 and the banner updated 2026-10-22 · sections 01–11 are the August record

Nothing in this document describes a real-world deployment. Every figure is measured on benchmark footage, held-out source videos, or spliced reconstructions during development and testing — not on incidents an installed system has ever actually observed. "Configured on" means the detector is the one the app currently ships and runs with, not that it has been field-validated.

Unfamiliar term? §10 defines every one used here — FPR, recall, precision, leakage, EMA and the rest.

---

00 The system today (22 October 2026)

The sections after this one were written in August and describe the August models — read them for *why*, not for the current numbers. Since then the violence detector became two models averaged, weapons moved to a higher-resolution model, vandalism was re-enabled, and a **second AI now double-checks every alarm** before anyone is notified. Every figure below is on footage the models never trained on: 5 hours of held-out street cameras for false alarms, and held-out crime clips spliced into continuous footage for events caught.

FIGHTS, WITH THE DOUBLE-CHECK ● IN THE APP

94.0%

of 200 held-out fights, at 0.4 false alarms per hour (88.0% at 1.4/hr without it).

ROBBERY, WITH THE DOUBLE-CHECK ● EXPERIMENTAL

59.4%

of 133 events labelled as robbery, at 0.6/hr (8.3% without it). Still the weakest class.

ARMED-THREAT FALSE ALERTS ● IN THE APP

1.4/hr

down from 2.8/hr, while keeping 97.9% of real weapons.

DETECTOR	WITHOUT THE DOUBLE-CHECK	WITH IT (GPU)	WITH IT (CPU, 4 GB LAPTOPS)
Violence	88.0% at 1.4/hr	**94.0% at 0.4/hr**	91.0% at 0.2/hr
Robbery (experimental)	8.3% at 1.4/hr	**59.4% at 0.6/hr**	60.9% at 0.8/hr
Weapons (alerts, not detections)	2.8 false/hr	**1.4 false/hr**, 97.9% of real weapons kept	not checked yet
Vandalism (experimental)	87.1% of 31 acts at 0.6/hr — not double-checked		

What the double-check is

A small vision-language model, **Qwen3-VL-2B**, is shown the last two seconds of an alarm (for weapons, a crop around the person holding it) and asked a plain question — "is anyone physically fighting?", "is someone being robbed?", "is this person holding a gun or a knife?". It is **not retrained**: only its pass/fail cut-offs were fitted, on half the source videos and scored on the other half. Because it removes false alarms, the fast detectors can run looser and miss less — that is where the gain comes from.

Robbery is asked two questions, because one yes/no question passed 86% of real *fights* as robbery: is a crime happening, and is it more like a robbery than a fight. A robbery confirmed within 60 seconds of an assault alert upgrades that one incident instead of opening a second.

> **Limits, said plainly.** A check delays the alert by about 7 s (GPU) or 15 s (CPU). It needs memory: on a 4 GB graphics card it runs on the CPU, and with too little free RAM it stays off and the strict August-style settings apply — nothing crashes. Weapon alerts are only double-checked on the GPU backend so far. The weapon figure rests on 14 alerts in 5 hours, so read it as "about half". And, as everywhere in this document, recall on the actual deployment cameras is unmeasured.

01 The one-page answer

Three detectors were built. Two ship. The third is switched off in `config.json` with its measurements recorded beside it, because both routes to it were tried and both failed — and a detector that alarms on seven of eight quiet clips is worse than no detector at all.

VIOLENCE ● SHIPPED

95.0%

38 of 40 events on continuous footage. 17 alarms/hour across three cameras.

ROBBERY ● SHIPPED

65.3%

of clips in eight unseen scenes, at 86.5% precision and 5.6% false-positive rate.

VANDALISM ● DISABLED

0%

The shipped rule has never fired once — 0 of 40 real clips.

> **The one thing to say before anyone asks.** Recall on the actual deployment cameras is **unmeasured for every class**, because no labelled incident has ever been recorded on them. Everything below is measured on benchmark footage, held-out source videos, or spliced reconstructions. That gap is a property of the problem — you cannot label violence that has not happened on your pole — not an oversight.

02 How the violence detector behaves

> **Checkpoint update since this section was written (2026-08-18).** The checkpoint changed from `corpus_neg` to `daynight`. Everything below this note — the confirmation sweep, the per-camera bars, the false-alarm journey, the confidence-trace animation in §8b — was measured on `corpus_neg` and has not been re-run against `daynight` using the same protocol; re-stating those specific numbers as daynight's own would mix two different measurement methodologies. What *can* be stated on equal terms, from the checkpoint's own deployment record: recall is unchanged (95.0%, 38 of 40 events on the same spliced test), and real-camera false alarms fall from an aggregate 12.75/hr to 4.50/hr. That is not a uniform win — it is nearly all the flyover camera (45 → 6/hr); the camera outside Lyn's Restaurant gets **worse** (6 → 12/hr). On its own held-out test split (1,310 clips, a different split from the numbers below), the new checkpoint scores 90.1% accuracy and a 3.7% false-positive rate. Say the Lyn's regression out loud if asked why every number did not simply improve — it did not, and knowing that is more convincing than a chart implying it did.

The model is an **X3D-XS 3D convolutional network** — about 3 million parameters — that reads a 13-frame clip of the *whole frame* and returns a probability. It does not look at people individually. It has no concept of a punch. It learned that a certain kind of rapid, clustered, full-body motion looks different from a street where nothing is happening.

That distinction matters for the defense, because it predicts the failure mode exactly: **anything that makes the whole frame move quickly looks like violence to it.** A camera that pans does this. A camera on a flyover with traffic does this. That is not a bug to be tuned away — it is what the representation is.

The confirmation logic is what makes it usable

A raw classifier firing on every clip would be unusable. Between the model and an alert sits a smoothing and confirmation stage: an exponential moving average, then a requirement that the smoothed score stay above threshold for *N consecutive* inference passes before an alert is emitted.

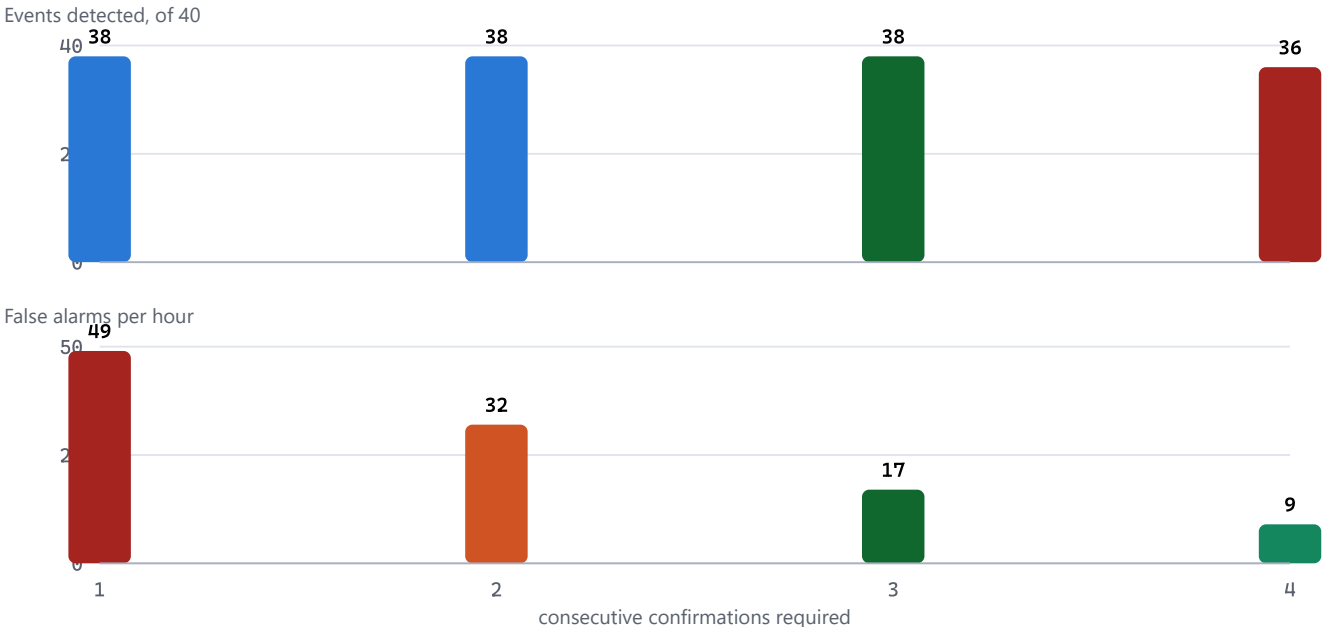
For most of this project that parameter was set to 1 in deployment while every published false-alarm figure had been measured at 2 — so the real system was roughly 53% noisier than the numbers said. Finding that mismatch produced the single largest improvement in the project, and it cost nothing:

Raising the confirmation requirement is free until it isn't

40 held-out violent clips spliced into real Davao night footage, 25 s of ordinary activity either side, replayed through the shipped state machine

The setting	How many times in a row the model must say “violence” before the system raises an alert.
Top panel	Of 40 real incidents, how many the system caught. Higher is better.
Bottom panel	How often it cried wolf on quiet footage. Lower is better.

What to look for: the top bars stay flat at 38 while the bottom bars fall by two thirds — the system got dramatically quieter without missing anything more. That holds until 4, where the top bar finally drops.



Three is the last free step; four is the first that costs. Going from 1 to 3 removed 65% of false alarms and lost none of the 40 events. Going to 4 saves 8 more alarms per hour and starts missing real ones. The two measures are drawn as separate panels on purpose — they have different units, and putting them on one pair of axes would be a chart that lies.

The number that decides whether anyone keeps it switched on

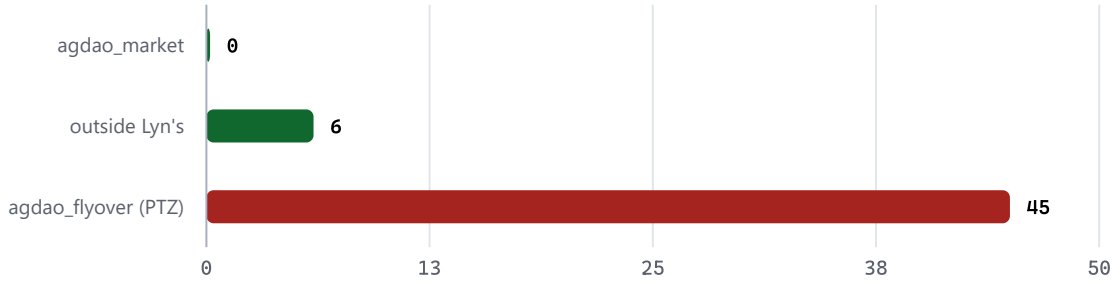
A system-wide false-alarm average would be actively misleading here. The spread between the best and worst camera is more than **45×**:

False alarms per hour, by camera

Threshold 0.50, confirmation 3, 20 minutes measured per camera

Each bar	One real camera, and how many times per hour it alerted when nothing was happening.
Why it matters	False alarms are what make an operator switch a system off. This is the number that decides whether it survives contact with a real control room.

What to look for: these three are the same model with the same settings — the only thing that changed is where the camera points. A market camera is silent; a camera on a flyover with moving traffic alerts 45 times an hour. **Performance is a property of the camera, not just the model.**



The flyover is the worst case and also the most trustworthy figure. It is the only one of the three whose venue does not also appear in training. The two quiet cameras each have a different camera at the same venue in the training set — no camera is in both roles, verified frame by frame, but venue-level leakage and "we added market negatives and market cameras improved" predict the same ranking and **cannot be separated with this data**. Quote the 45, not the 17, when you want to be safe.

How it got here — the two things that actually worked

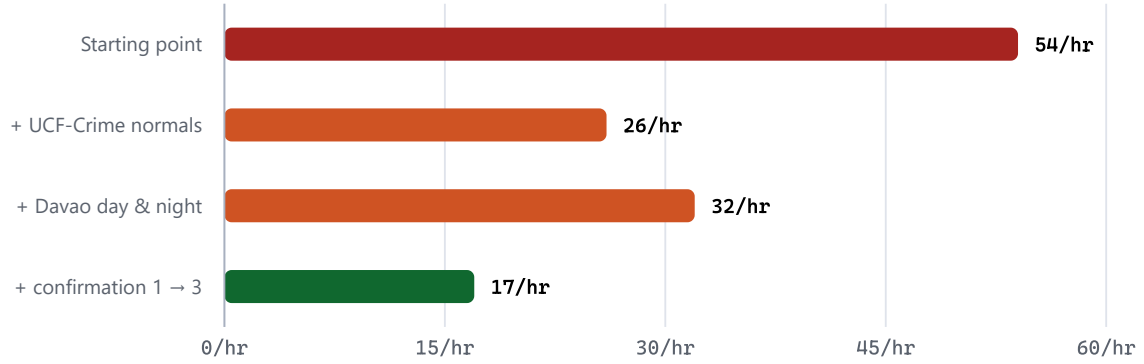
The violence detector did not start at 95%. It started at **78.4%**, and on real street footage it alarmed **54 times an hour** — roughly once a minute, on empty pavement. Two interventions fixed it, and neither was a change to the model:

The false-alarm rate, milestone by milestone

Measured on the same real Davao street cameras throughout, so the steps are comparable

Each bar False alarms per hour after one change. Lower is better.

What to look for: nothing here is a smarter model. Two of the steps are **feeding it more examples of ordinary life**, and one is **fixing a setting that had been measured wrong for the whole project**. The architecture never changed.



The last step is the uncomfortable one. Every published false-alarm figure had been measured at a confirmation setting of 2, while deployment actually ran 1 — so the live system was about 53% noisier than the numbers claimed for most of this project's life. Finding that mismatch, and then setting it to 3, produced the single largest improvement here and cost nothing in detection.

The accuracy story runs alongside it, and one entry is a bug rather than an improvement: the model's output was being passed through a softmax *twice*. Because softmax over a pair can only produce values between 0.269 and 0.731, **every confidence this system had ever reported was squeezed into that narrow band** — 79% of clips came back as one of exactly two values. Thresholds tuned against those numbers were tuning against a distortion.

STAGE	ACCURACY	WHAT CHANGED
After leakage removal	78.4%	The honest starting point — earlier figures counted clips from videos that were also in training
Double-softmax removed	–	Confidences stopped being compressed into a 0.46-wide band; no verdict changed at 0.50, but every other threshold became meaningful
Group-aware 3-way splits	–	Splitting by source video, so no scene appears on both sides
Current	**95.0%**	On a test split touched by neither training nor checkpoint selection — unchanged by the corpus_neg → daynight update (see note above)

> **Why the numbers went up when the model did not change.** Most of this gain is *honesty*, not capability: removing leaked clips first made the score drop, and every later figure is measured on data the model has genuinely never seen. A number that goes up because the measurement got stricter is worth more than one that goes up because the test got easier.

Where it goes blind — and why that is the dangerous failure

Replaying 40 clips the model detects at full confidence, while progressively shrinking the people inside them, maps the edge of its vision. Below roughly 15% of frame height it does not become less confident. It stops seeing anything at all.

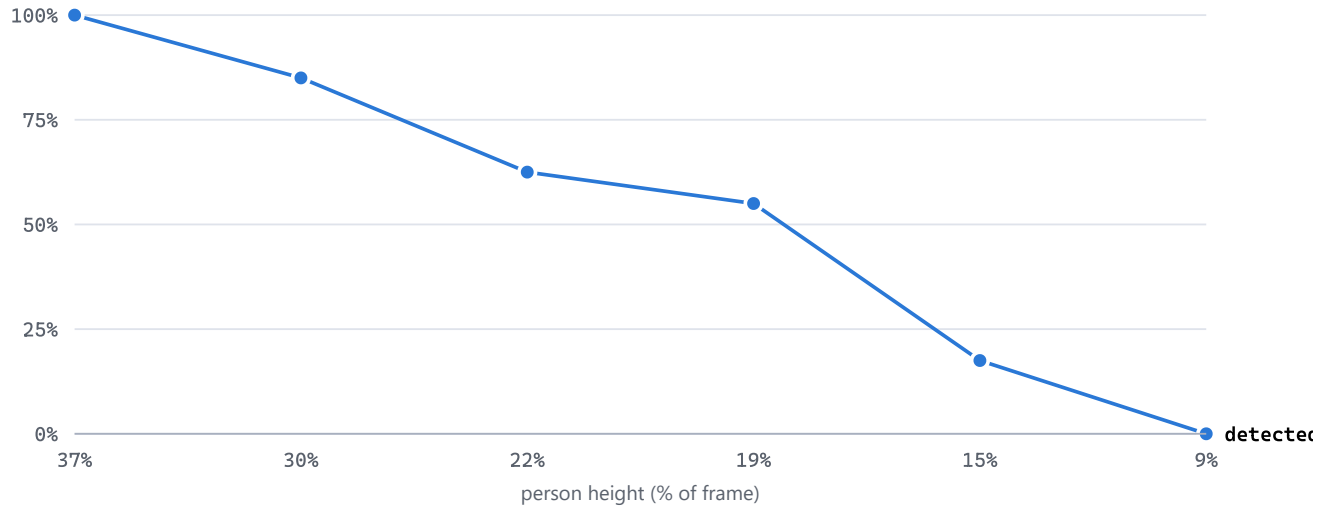
Detection collapses with subject size

Same 40 clips, rescaled. Person height as a percentage of frame height

Left to right The same fights, with the people made progressively smaller in the frame — as if the camera were mounted further and further away.

The line How many of the 40 it still recognised at that size.

What to look for: this is not a gentle decline. The model holds up to about 22%, then falls off a cliff and reaches **zero** by 9%. There is a distance past which it sees nothing at all — and it gives no signal that it has stopped working.



A blind camera and a safe street produce identical output: zero alarms. For a public-safety system that is the most dangerous failure mode there is, because nothing looks wrong. This chart is the reason the project measured actual subject size on the deployment cameras rather than assuming it — see §4.

Raw counts, on equal terms with daynight

Unlike the charts above, this one *is* measured on the current (daynight) checkpoint — a fresh run, not carried over from corpus_neg. One prediction per clip, at the threshold the app is actually configured with (0.50), on a 1,310-clip held-out test split.

Confusion matrix — violence, daynight checkpoint

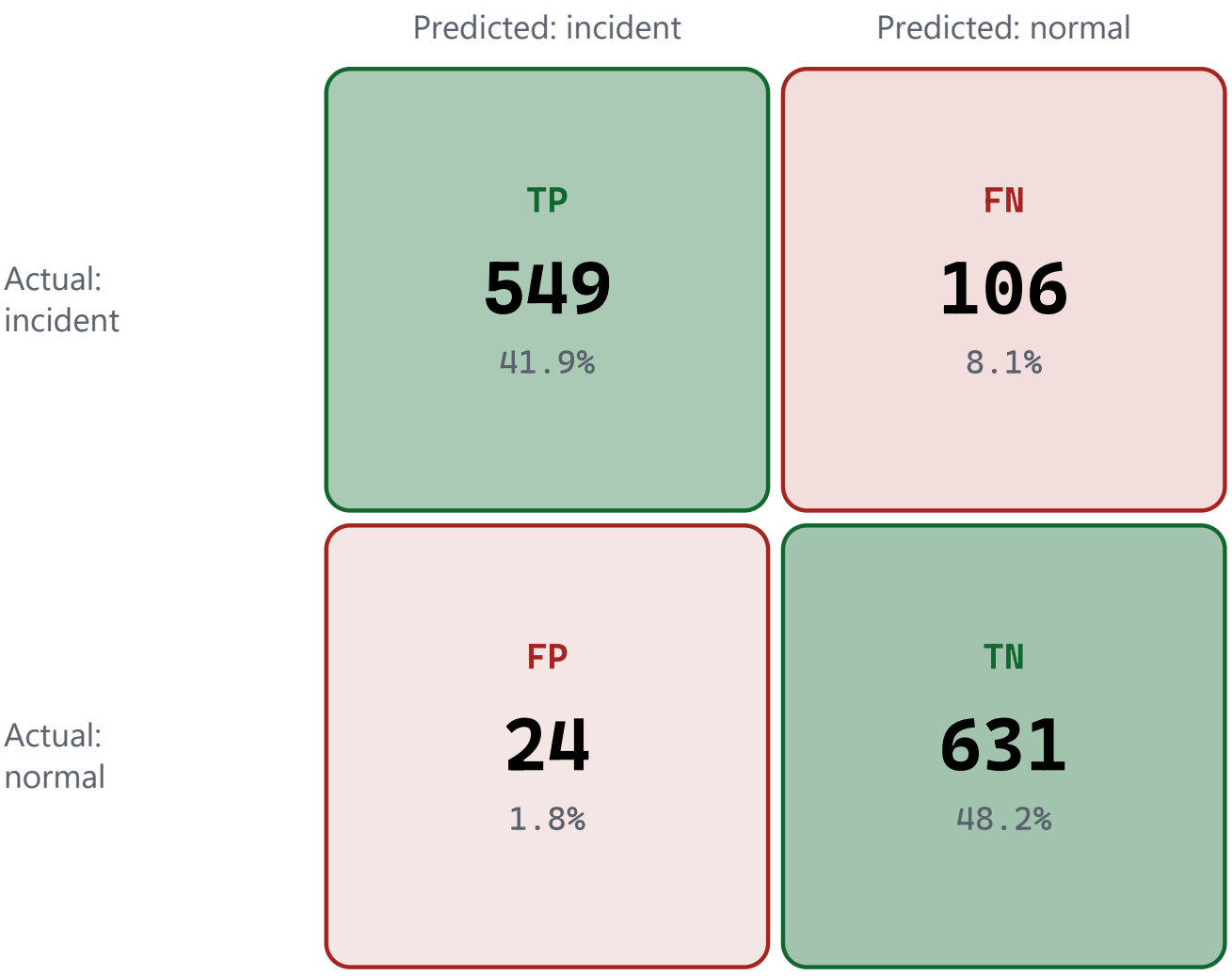
Threshold 0.50 · 1,310 held-out clips, one prediction per clip

TP / TN The system got it right — flagged a real incident, or correctly stayed quiet.

FN A real incident the system missed entirely.

FP A false alarm on ordinary footage.

What to look for: TP and TN are the two green quadrants and should be the bulk of the grid. FN (106 clips, 8.1%) is the number that matters most for a safety system — it is larger than FP (24 clips, 1.8%), consistent with this detector being tuned toward precision rather than recall.



Accuracy 90.1% · precision 95.8% · recall 83.8% · false-positive rate 3.7%. Recall here (83.8%, per-clip) reads lower than the 95.0% headline event-detection figure elsewhere in this document — they are different measurements. This one scores every clip in isolation; 95.0% scores whether a continuous, multi-frame incident was caught at all, which is more forgiving of a single missed frame inside an event the system otherwise caught. Both are real; they answer different questions.

03 How the robbery detector behaves

Same architecture, different training set, and one design decision that made the class possible at all.

The negatives are the **non-crime spans of the same videos as the positives**. A shop interior therefore appears on both sides of the label, and the model cannot separate the classes by recognising the DVR overlay, the lighting, or the room. It has to separate them on the act. That is what makes indoor footage safe to include, and including it took the class from 15 usable source videos to **43**.

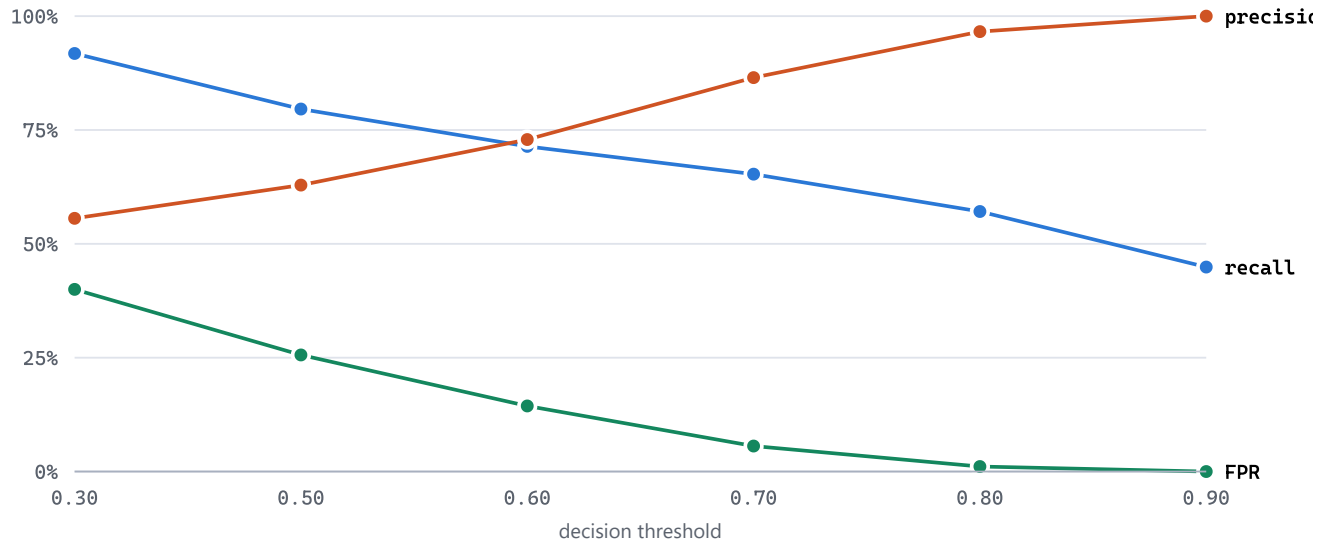
Splitting is by *source video*, never by clip: 26 train / 9 val / 8 test scenes, zero group-level leakage.

Choosing the operating point

139 clips from 8 source videos the model never trained on

Threshold	How confident the model must be before it calls something a robbery. Left = trusting, right = strict.
Recall	Of the real robberies, the share it catches. *Miss rate is the rest.*
Precision	Of the alerts it raises, the share that are genuine. *The rest are false alarms.*
False-positive rate	How often it alerts on ordinary footage.

What to look for: the two lines move in opposite directions, and that trade is unavoidable — you cannot raise both. Being stricter catches fewer real robberies but wastes far less of an operator's attention. **0.70 is the point chosen, and choosing it is a judgement about which mistake costs more**, not something the data decides on its own.



0.70 is where precision becomes usable without recall collapsing. At 0.50 a quarter of normal clips misfire — the alert-fatigue failure this project spent most of its effort on for violence. Past 0.80, precision is nearly perfect and recall falls off a cliff. The chosen operating point accepts missing about one robbery in three in exchange for a 5.6% false-positive rate.

> **Say the recall number out loud before the panel finds it.** 65.3% means it misses roughly one robbery in three. That is defensible for a first version built on 43 source videos, and it is deliberately precision-weighted — but it is not a "catches everything" claim and should never be presented as one.

Two further caveats belong with any figure quoted from this table. The test set is **8 scenes, not 139 independent samples** — clips from one video are highly correlated. And the confirmation requirement of 3 is **carried over from violence, not tuned**, because there is no continuous robbery footage to tune it on.

What it replaced is worth stating plainly: a hard-coded constant, `0.895`, with no derivation anywhere, which the dashboard rendered as though it were a model output.

▾ Full threshold sweep — table view

THRESHOLD	ACCURACY	RECALL	PRECISION	FPR
0.30	71.2%	91.8%	55.6%	40.0%
0.50	76.3%	79.6%	62.9%	25.6%
0.60	80.6%	71.4%	72.9%	14.4%
0.70 — current	84.2%	65.3%	86.5%	5.6%
0.80	84.2%	57.1%	96.6%	1.1%
0.90	80.6%	44.9%	100.0%	0.0%

Confusion matrix — robbery, at the configured threshold

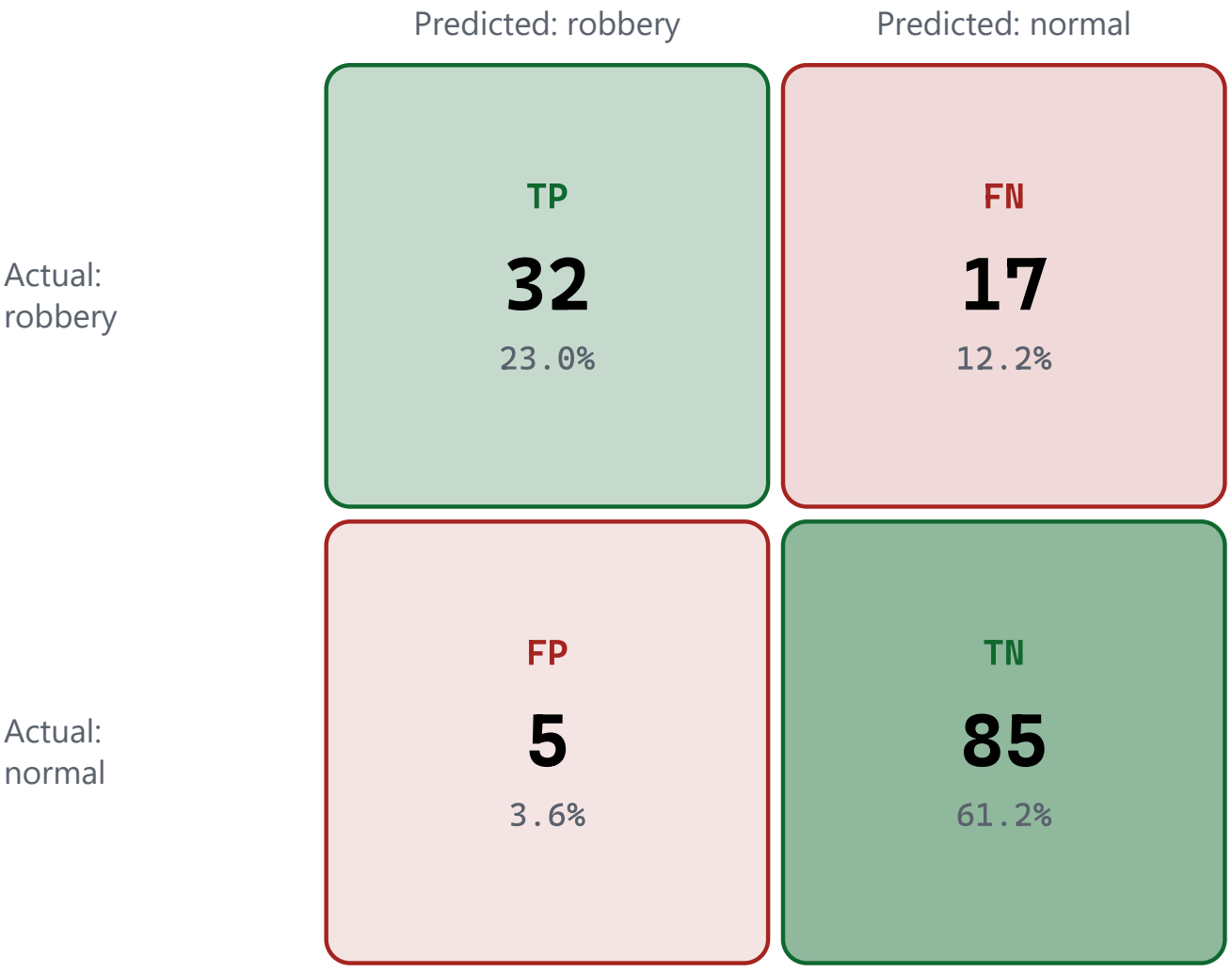
Threshold 0.70 · 139 clips from 8 source videos never trained on

TP / TN The system got it right — flagged a real robbery, or correctly stayed quiet.

FN A real robbery the system missed.

FP A false alarm on ordinary footage.

What to look for: FN (17 clips) outweighs FP (5 clips) by more than 3 to 1 — the direct visual of "misses about one robbery in three," stated plainly in §3 above. This is a design choice, not an accident: 0.70 was chosen specifically to keep FP low.



Accuracy 84.2% · precision 86.5% · recall 65.3% · false-positive rate 5.6%. Remember the caveat from §3: this is 8 source scenes, not 139 independent samples — clips from the same video are correlated, so treat this grid as indicative rather than statistically tight.

04 Vandalism: why it is switched off

Two independent routes were built and measured. Both failed, in different ways, and the honest outcome was to disable the class rather than ship something that produces confident-looking output.

Route one — the rule has never fired

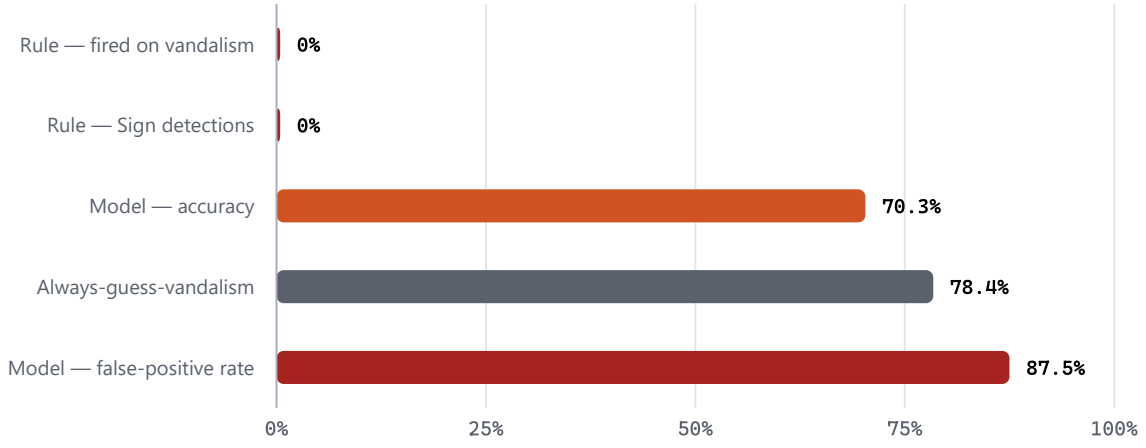
`score_vandalism()` requires a person's wrist to be near a YOLO-detected *sign* box. Given the first labelled vandalism footage this project has ever had, that gate opened **zero times**:

Both routes, measured

Left: the shipped rule on 64 labelled clips. Right: the trained model against the majority-class baseline

First two bars	The hand-written rule. It never fired, and the thing it waits for never occurred.
Model accuracy	How often the trained model was right.
Always-guess baseline	How often you would be right by ignoring the video entirely and answering "vandalism" every time. **This is the bar any model must clear to be worth anything.**

What to look for: the model's bar is **shorter** than the do-nothing baseline. It is not a weak detector — it is worse than not looking, and its false-alarm bar is the tallest on the chart. Both routes failed independently, which is why the class ships switched off.



Zero `Sign` detections across 4,800 sampled frames. Person tracking worked normally in 1,260 of those frames, so the pipeline is healthy — the condition is simply not satisfiable. This is not a tuning problem: no threshold makes a detector that never fires work. And the trained model's 70.3% accuracy is **below the 78.4%** you get by labelling every clip vandalism, so it carries negative information.

The trained model fires on **7 of the 8 normal clips** in its test set. Its training curve confirms what that means: validation accuracy fell from 42% to 33% while training accuracy climbed to 86% — it memorised eleven scenes.

> **The blocker is scene count, not algorithm.** Robbery works at 26 training scenes. Vandalism fails at 11. Filtering existing corpora cannot cross that gap, because the limit is *distinct locations* and filtering does not create any. Filming does. The route across is roughly 20 outdoor scenes, documented in `docs/vandalism_data_collection.md`.

Confusion matrix — the trained model, at its least-bad threshold

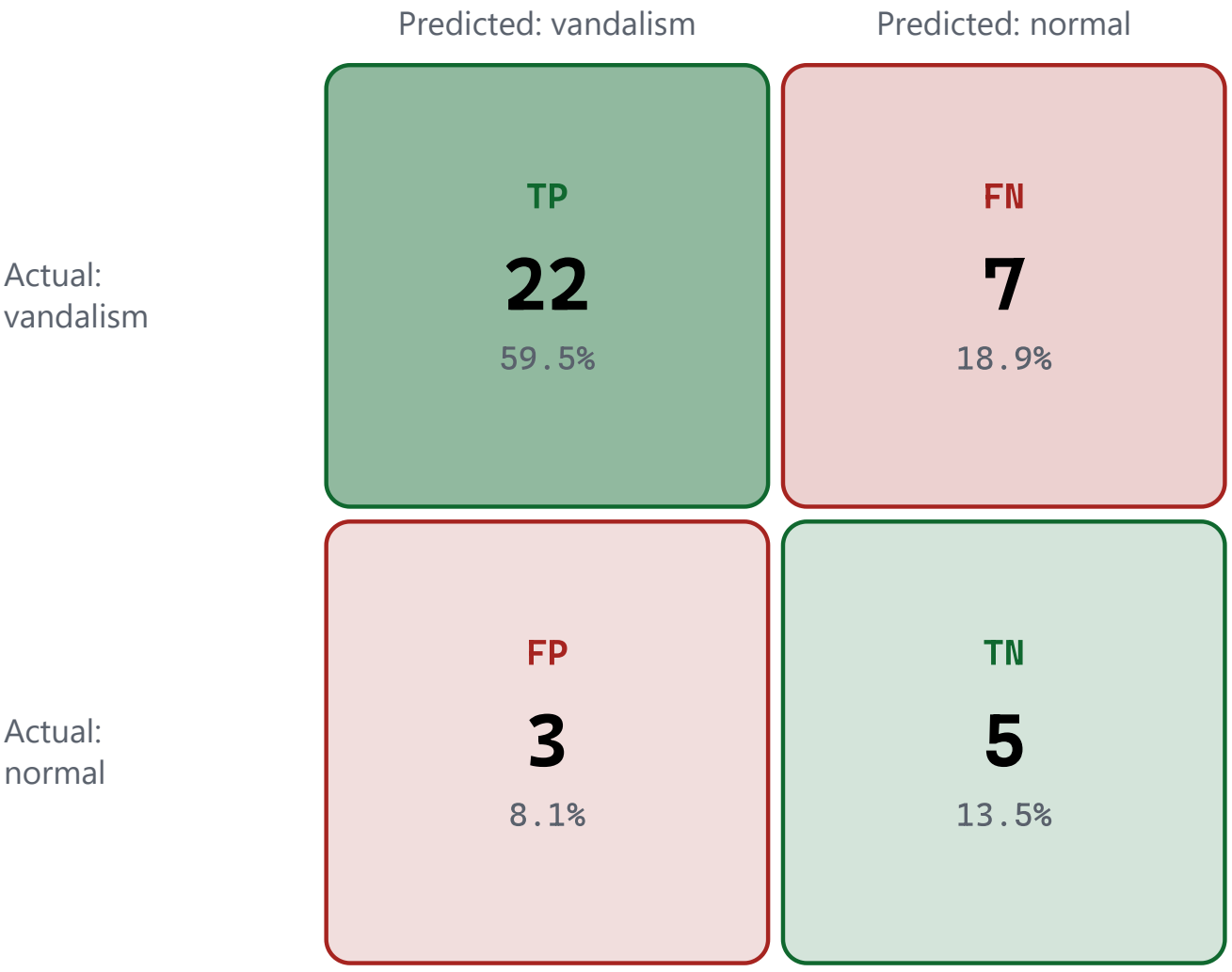
Threshold 0.90 · 37 clips (29 positive, 8 negative) — the entire test set

TP / TN The system got it right.

FN A real vandalism clip it missed.

FP A false alarm on ordinary footage.

What to look for: only 8 negative clips exist in this test set at all, so FP + TN = 8 total — each single clip is worth 12.5 percentage points of false-positive rate. This grid is the reason the class ships disabled, not a table of percentages that hides how thin it is.



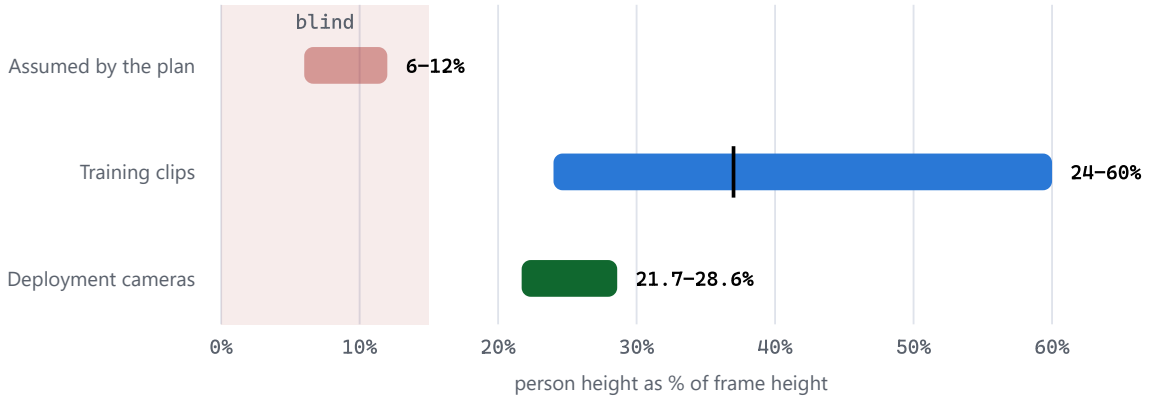
Accuracy 73.0% · precision 88.0% · recall 75.9% · false-positive rate 37.5% — at 0.90, the least-bad threshold found. At the 0.50 used elsewhere in this document for comparison, the same 8 negative clips misfire 7 of 8 times (87.5% FPR); 0.90 is already the improved number, and it is still not good enough to ship.

05 The assumption that was wrong

This project's plan was written around the belief that a wide street camera puts a person at 6–12% of frame height — comfortably inside the blind region from §2. Measuring 248 person detections across the four deployment cameras showed it is not true.

Subject size: assumed, trained, and actual

Person height as a percentage of frame height. Measured with `measure_object_scale.py`



The deployment cameras sit at 22–29%, close to the training distribution — not at 6–12%. The scale problem is far less severe than the project believed for most of its life, which is why cropping and tiled inference were dropped. **The caveat runs the other way, though:** these are heights of *detected* people, and YOLO misses small distant ones by definition, so the true distribution has more small people in it than this and these medians are biased upward.

The same measurement answers whether object recognition would help. Against a roughly 24-pixel floor for reliable small-object detection: a backpack is about 38 px and **detectable**; a carried box is 29 px and borderline; a **phone or wallet is 10 px — below detection**. A "detect the stolen item" feature is not available at these camera distances, regardless of model choice.

06 What was tried and lost

This is the most transferable part of the work, and it should be presented as a finding rather than hidden. **Every algorithmic intervention lost.** What worked was more representative data and correctly configuring a temporal parameter that had been mismeasured for the entire project.

INTERVENTION	OUTCOME
Pose gate — require ≥2 people	Lost to a 0.05 threshold change
Wrist-velocity gate	Lost
Motion-localisation gate	Lost
Tiled inference, 4×4 with 25% overlap	Lost at every scale, at 8× the compute
Flyover artefact theories — scale, global motion, blur, pan/zoom	All four *depleted* among alarms
"Short UCF videos are mostly anomaly"	Median coverage 0.32 — an upper bound
"Motion localises the anomaly"	Crime is busier in 11 of 21 sources, quieter in 10
Object-centric features (COCO-80)	+0.12% over published RTFM — inside the noise
More and more diverse negatives	**54 → 26 → 17 alarms/hour**
Confirmation requirement 1 → 3	**49 → 17 alarms/hour, at no cost in recall**

---

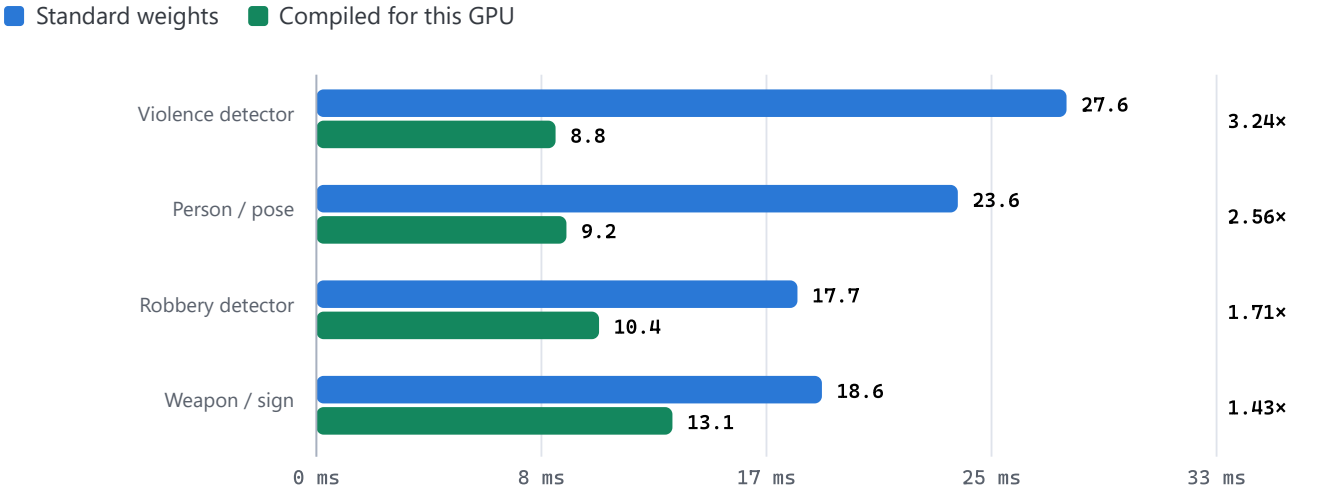
07 What ships, and how fast it runs

Four models, all local. No cloud service, no internet at run time, no external database. The whole pipeline holds about **295 MB of GPU memory** including the CUDA context, and 1.22 GB of system RAM.

The installer offers an optional step that recompiles each model for the GPU it finds. This has to happen on the target machine: a compiled engine is locked to the architecture that built it, so shipping them prebuilt would produce files that simply fail to load on any other card.

Optional GPU compilation, measured before and after

Milliseconds per inference on a GTX 1660 SUPER. Lower is better



Accuracy is unchanged, and that is verified rather than assumed. Each compiled model is diffed against the original before being installed; one that disagrees is discarded. Measured through the live detector: max deviation 0.000026 over 150 frames, and **zero verdict changes** at the configured threshold. Without that guarantee, every threshold in the config would quietly mean two different things depending on whether a compiled model happened to load.

---

08 Answers to the questions you will be asked

"Why not just use a bigger model?"

Because every accuracy gain in this project came from data, not capacity — see §6. Four separate fine-tuning runs peaked at epoch 1, 2, 2 and 4 before drifting, and the most recent day-plus-night retrain peaked at **epoch 1 and never beat it across seven epochs**. The models converge almost immediately; the remaining error is not something more parameters or more epochs reach.

"Your violence accuracy is 95% — why is robbery only 65%?"

They are not the same measurement. The 95% is a *detection rate for events* on continuous footage — did an alert fire during the incident. The 65.3% is *per-clip recall* across eight scenes. On the comparable per-clip measure the violence model scores 91.4% recall, and it has an order of magnitude more training data behind it.

"Why is vandalism in the system at all if it is disabled?"

Because the measurement is the contribution. The class is off, with both failure modes recorded in `config.json` next to the switch, and a documented route to

enabling it. A negative result that is written down is reusable; one that is quietly deleted is not.

"How do you know the model is not just memorising cameras?"

Splits are by source video, never by clip, so no scene appears on both sides. For robbery, the negatives come from the same videos as the positives, so camera identity carries no label information at all. For violence, the holdout cameras were checked frame by frame against the training set — and where a *venue* overlaps, §2 says so and points at the flyover as the clean figure instead.

"What would you do next?"

Multiple Instance Learning, in the Sultani formulation — video-level labels over 32 segments with a ranking loss. It removes the need for temporal annotation, which is the actual bottleneck: it would take robbery from 43 to roughly 400 source videos and vandalism from 18 to about 100. Every other lever in §6 has already been pulled and lost.

> **The data for this is already on disk.** MIL trains on *untrimmed* videos with a single label each — which is exactly what UCF-Crime ships as, and what this project already downloaded and then spent considerable effort cutting into hand-annotated clips. The 43 robbery sources are 43 because that is how many survived manual annotation; the full category is roughly ten times that. Nothing new needs collecting.

Where to get it

WHAT	WHERE	WHY THIS ONE
Sultani et al., 2018 Real-world Anomaly Detection in Surveillance Videos (CVPR)	arxiv.org/abs/1801.04264 github.com/WaqasSultani/AnomalyDetectionCVPR2018	The original MIL ranking-loss formulation, and the paper that introduced UCF-Crime itself. Start here — the method is simple enough to reimplement.
RTFM Tian et al., ICCV 2021	arxiv.org/abs/2101.10030 github.com/tianyu0207/RTFM	The standard modern baseline on UCF-Crime; feature-magnitude selection fixes MIL's weakness on subtle anomalies. This is the number other papers compare against.
MGFN Chen et al., AAAI 2023	arxiv.org/abs/2211.15098 github.com/carolchenyx/MGFN.	Magnitude-contrastive; competitive with RTFM and often easier to train on a small GPU.
UMIL Unbiased MIL, CVPR 2023	github.com/ktr-hubrt/UMIL	Directly targets the context bias this project fought by hand — the "indoor = violent" problem that forced UCF-Crime's violence half to be excluded.
WSAD collection	github.com/Minqi824/WSAD	Curated index of weakly-supervised anomaly detection work; the fastest way to see what has already been tried.
UCF-Crime, untrimmed	crcv.ucf.edu/projects/real-world/	Already downloaded for this project. MIL wants the full-length videos, not the clips.

arXiv identifiers are from memory — verify each before citing in the paper. The GitHub paths are the ones to search on if a link has moved.

"What is the second AI, and why not just train the first one better?"

Qwen3-VL-2B is a general video-language model used as it was released — nothing about it was retrained. The fast detectors have to run on every frame of every camera, so they are small; one frame cannot tell a robbery from an ordinary hand-off in a busy market. The second model is too slow to run constantly (7 to 15 seconds per look) but only has to look at the few clips the detectors flag. A robbery retrain and a bigger verifier (Qwen3-VL-4B, no better and four times slower) were both measured and rejected.

"Does it work on a laptop?"

Yes, in one of three modes picked automatically: on the graphics card if 4.3 GB is free, otherwise on the processor (about 2 GB of RAM, slower but nearly as accurate: 91.0% of fights at 0.2/hr), otherwise off. The test laptop (RTX 3050 with 4

GB, 8 GB of RAM) uses the processor mode, which keeps the camera AI at about 85% of its speed.

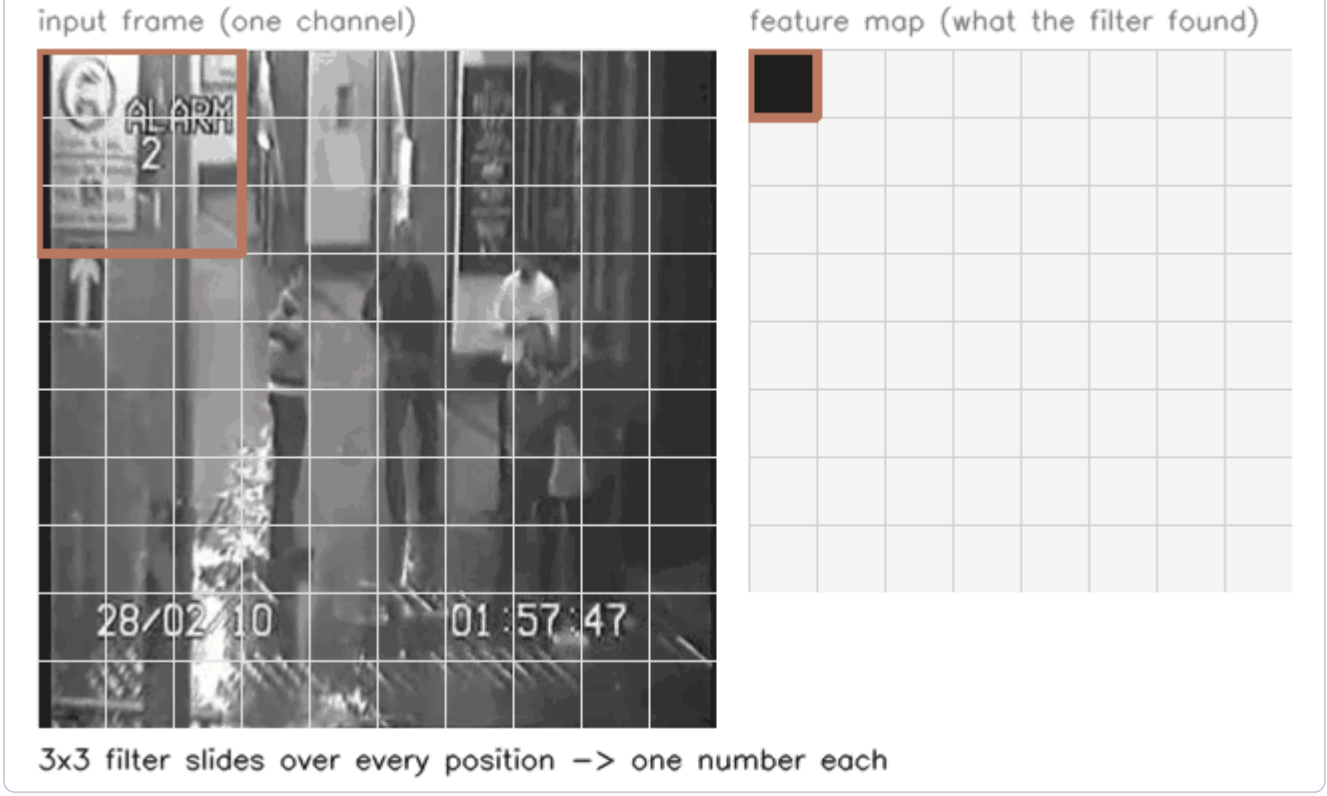
"Isn't the AI making the decision?"

No. A person always confirms or dismisses an alert on the dashboard, and the double-check only decides whether an alarm is worth a person's attention. If it is unavailable, every detector falls back to its stricter settings.

08b What the network actually does

Three animations, all built from one real CCTV clip in the dataset and from the checkpoint that was current when these were built (corpus_neg, since superseded by daynight — the underlying mechanics shown here do not change with a fine-tune). Nothing here is a textbook diagram.

A convolution is a filter that slides



A 3×3 filter visits every position in the frame. At each stop it multiplies nine weights against the nine pixels beneath it and produces **one number**. Those numbers build the map on the right. This filter responds to vertical edges, so the map brightens wherever one appears. **That is the entire operation** — a network is this, repeated, with later layers finding combinations of what earlier layers found.

The 3D version reaches across time — and that is the whole point



The filter also spans **three frames at once**, so the number it produces describes how the picture *changed* rather than what it contained. This is why an image classifier cannot do this job: a single frame of a punch and a single frame of a wave look nearly identical, and the thing that separates them exists only *between* frames.

And this is what comes out



The corpus_neg violence checkpoint (current when this trace was recorded; see the update note in §2) on a held-out CCTV fight clip. The trace is its own confidence over time, with the 0.50 threshold marked. **Watch the gap:** confidence crosses the threshold several frames before the alert fires. That delay is the confirmation rule from §2 doing its job — and it is what took the system from 49 false alarms an hour to 17.

---

09 How the system works, step by step

> **One step added since this was written.** After a detector raises an alarm and before an alert is sent, the clip goes to the Qwen3-VL-2B double-check (section 00). The steps below describe the detectors themselves.

In the order a single frame travels through it. Every name below is a real function in the repository, so a question about any step has somewhere to land.

STEP	CODE	WHAT IT DOES, AND WHY IT EXISTS
1. Read the camera	`_open_capture` `_NetworkStreamReader`	Opens a webcam, file or RTSP stream. Network streams get a background reader thread that always keeps the *newest* frame and drops the rest — a network camera delivers faster than the pipeline consumes, and without this the system would fall further behind real time the longer it ran.
2. Find people	`YOLO11s-pose` `load_model_with_fallback`	Detects each person and 17 body keypoints, and assigns a tracking ID that persists across frames. The loader prefers a GPU-compiled `.engine` and silently falls back to the ordinary `.pt` — so a machine without the compiled version still works, just slower.
3. Find weapons	`_run_weapon_detection` `VBoxTracker`	A second network looks for Gun, Knife and Sign. It runs on a *separate thread* because it is slower than the pose pass; blocking on it would drop the frame rate for everything. `VBoxTracker` then links detections across frames into tracks using IoU, so one knife is one object rather than a new detection every frame.
4. Discard scenery	`_is_static_scene_object`	Rejects any weapon track that has never moved. This exists because of a measured failure: the detector locked onto a utility pole and reported it as a Gun at 0.93 confidence, frame after frame — "6,438 detections/hour" that were one stuck box re-counted. A carried weapon moves; a pole does not. It removed 97.4% / 81.0% / 78.1% of detections on three real cameras. **The cost is stated, not hidden:** a genuinely motionless weapon is suppressed, and it is released the moment it moves.
5. Attribute weapons to people	`_assign_weapons` `_vbox_overlap_ratio`	Decides *who* is holding what, by comparing wrist keypoints and body boxes against weapon boxes. The assignment is sticky — once a weapon is attributed to a person it stays with them through brief occlusion, rather than flickering between bystanders.
6. Buffer the frames	`SceneViolenceDetector.update` `_prepare_clip_array`	Violence is motion, so a single frame cannot show it. Frames accumulate in a rolling buffer; 13 are sampled evenly across it, resized to 160×160 and normalised. This produces the short video clip the classifier actually reads.
7. Classify the clip	`_run_inference` `_forward`	The X3D network returns a probability that the clip contains violence. `_forward` chooses between the compiled and standard model — both produce identical numbers, verified to 0.000026 with zero verdict changes, which is what allows the choice to be invisible to everything downstream.
8. Smooth and confirm	`_smooth_and_confirm`	**The single most important function for usability.** Three jobs at once: an *exponential moving average* so one noisy reading cannot raise an alert; a *consecutive-hit counter* requiring several passes in a row to agree; and *sticky release*, which keeps an alert active until confidence drops

STEP	CODE	WHAT IT DOES, AND WHY IT EXISTS
		well below the threshold rather than at it — otherwise an alert would flicker on and off while a fight is still happening. Raising the consecutive requirement from 1 to 3 here removed 65% of false alarms at no cost in detection.
9. Decide and alert	`TrackState.update should_alert _post_alert`	Holds per-person state and enforces a *cooldown*, so one incident produces one alert instead of a stream of them. On firing it writes a snapshot, posts the incident to the backend, and the dashboard receives it over a websocket.
10. Optional: compile for this GPU	`_load_engine optimize_weights.py`	Recompiles the models for the exact GPU present, giving 1.4×–3.2×. Never shipped prebuilt, because a compiled model only runs on the architecture that built it. Any compiled model that disagrees with the original is discarded rather than installed.

> **Why so much of this is subtraction.** Steps 4, 8 and 9 all exist to *remove* output, not to find more. That is the shape of the problem: a detector that fires often is easy, and a detector an operator will leave switched on is the hard part.

---

10 Every term used here

Plain definitions, with what each one means *for this system specifically*.

TERM	STANDS FOR	WHAT IT MEANS HERE
FPR	False-Positive Rate	Of the clips where *nothing* is happening, the share the model wrongly flags. 5.6% means about 1 in 18 quiet clips raises a false alert. Lower is better. **This is the number that decides whether anyone keeps the system switched on.**
Recall	—	Of the real incidents, the share the model catches. 65.3% means it misses about one in three. Also called sensitivity or the true-positive rate.
Precision	—	Of the alerts it raises, the share that are genuine. 86.5% means roughly 1 alert in 7 is a false alarm. **Precision and recall trade against each other** — you cannot raise both by moving the threshold.
Accuracy	—	The share of all clips judged correctly. Misleading on its own when the classes are unbalanced — the vandalism model's 70.3% accuracy is *worse* than guessing one class every time (78.4%).
Base rate	—	What you would score by ignoring the video and always answering the most common class. Any model must beat this to be worth anything at all.
Threshold	—	How confident the model must be before something counts as a detection. Lower catches more and cries wolf more.
EMA	Exponential Moving Average	A running average weighted toward recent readings. Smooths out single-frame noise so one odd reading cannot raise an alert.
Hysteresis	—	Using a *lower* bar to stop alerting than to start. Without it an alert flickers on and off while confidence hovers at the threshold.
X3D	Expanded 3D network	The video classifier. Ordinary networks read one image; this reads a stack of 13 frames, so it can see *motion*. The XS variant is the smallest — about 3 million parameters.
YOLO	You Only Look Once	The object detector, used twice: once for people and pose, once for weapons.
Pose / keypoints	—	17 estimated body joint positions per person — wrists, shoulders, hips. Used to decide who is holding a weapon.
IoU	Intersection over Union	How much two boxes overlap, 0 to 1. Used to recognise that a box in this frame is the same object as one in the last.
Track / track ID	—	One person followed across frames under a stable identifier, so state can be held per person.
Clip	—	A short fixed-length span of frames — the unit the classifier reads. Not the same as a video.

TERM	STANDS FOR	WHAT IT MEANS HERE
Scene / source video	—	One camera at one location. Many clips come from one scene, so **counting clips overstates how much independent evidence you have.**
Leakage	—	The same scene appearing in both training and test data. Produces high scores that mean nothing, because the model has effectively seen the answers.
Group-aware split	—	Dividing data by *source video* rather than by clip, so leakage is impossible by construction. Used everywhere here.
Held-out / test split	—	Data withheld from both training and model selection. The only honest measurement.
Validation split	—	Data used to pick which checkpoint to keep. Because it drives that choice, its score is *optimistic* and should never be quoted as final.
Overfitting	—	Memorising the training examples instead of learning the pattern. Visible as training accuracy rising while validation accuracy falls — exactly what the vandalism model did.
Softmax	—	Converts raw network output into probabilities summing to 1. Applying it twice by accident compressed every confidence this system reported into a narrow band.
Fine-tuning	—	Starting from a network already trained on a large general dataset and adjusting part of it for this task. Far cheaper than training from scratch.
Frozen / unfrozen layers	—	Frozen layers keep their existing values; unfrozen ones learn. Here the last 2 blocks are unfrozen — enough to adapt, not so much that it memorises.
Augmentation	—	Randomly altering training clips — flips, crops, scale — so the model sees more variety than was filmed.
MIL	Multiple Instance Learning	Training from a label on a *whole video* instead of on marked start/end times. The proposed next step, because manual time-marking is the real bottleneck.
PTZ	Pan-Tilt-Zoom	A camera that can move. Motion of the camera itself looks like motion in the scene, which is why the PTZ flyover is the noisiest of the three cameras.
TensorRT / .engine	—	NVIDIA's model compiler, and the file it produces. Faster, and locked to the exact GPU that built it.
VRAM	Video RAM	Memory on the graphics card. This whole pipeline uses about 295 MB of it.
Inference	—	Running a trained model to get an answer, as opposed to training it.

TERM	STANDS FOR	WHAT IT MEANS HERE
`UCF-Crime, RWF-2000, SCVD`	—	Public surveillance datasets used for training and benchmarking.

---

11 How the models were trained

Last, because nothing above depends on it: the shipped system only runs inference. This is how the files in `weights/` came to exist.

The recipe

All three detectors share it — only the data differs, which is deliberate. When one class works and another does not, having identical training means the difference is **the data, not the method**, and that is a claim worth being able to make.

CHOICE	VALUE	WHY
Starting point	`X3D-XS, Kinetics-pretrained`	Already understands general human motion. Training a video network from scratch on this much data would not work.
Layers unfrozen	`last 2 blocks`	Enough to adapt to CCTV, few enough to resist memorising. An early robbery run accidentally unfroze **nothing** — 4,098 of 2,978,772 parameters — and scored 11.9% recall. That run was discarded.
Input	`13 frames, 160×160`	Recorded in a `.meta.json` beside every checkpoint, which *overrides* config.json — geometry is a property of the weights, and config has no way to be right about it.
Loss	`NLLLoss(log(p))`	The network head already outputs probabilities, so the loss takes their log rather than applying softmax again — the bug described in §2.
Splitting	`by source video`	Never by clip. This is what makes the reported numbers mean anything.
Checkpoint choice	`best validation accuracy`	And therefore optimistic — which is why every headline figure in this document comes from the *test* split instead.

Where the training data came from

POOL	CLIPS	SCENES	ROLE
RWF-2000, SCVD, CCTV-Fights	3,413	hundreds	violence positives
UCF-Crime normals	900	150	violence negatives
Davao live capture, day	2,560	26	violence negatives
Davao live capture, night	1,320	22	violence negatives
UCF robbery + same-camera normals	795	43	robbery
UCF vandalism + same-camera normals	215	18	vandalism (disabled)

Two data decisions worth defending

UCF-Crime's violence half was deliberately excluded. About 96% of it is indoor. On an outdoor street camera, training on it teaches “indoor equals violent” — a shortcut that scores well on the benchmark and fails completely in deployment.

Robbery's negatives are the non-crime spans of the same videos as its positives. A shop interior therefore appears on both sides of the label, so the model cannot separate the classes by recognising the room, the lighting or the DVR overlay — it has to use the act. This is what made indoor footage safe to include, and it took the class from 15 usable sources to 43.

> **What training could not fix — and one update since this was written.** Four fine-tuning runs peaked at epoch 1, 2, 2 and 4 and then drifted. The day-plus-night retrain also peaked at **epoch 1** and never beat itself across seven epochs — at the time that looked disqualifying, so it was not adopted. It was adopted four days later anyway (2026-08-18): the epoch-1 peak turned out to be an artifact of comparing validation accuracy across two differently-calibrated manifests, not a real quality drop, and the checkpoint's own held-out test-split numbers (90.1% accuracy, 3.7% false-positive rate, 1,310 clips) held up once measured on equal terms. See the update note at the top of §2. These models converge almost immediately — the remaining error is not something more epochs or more parameters reach. It is scenes.

Scripts backing every figure: `sweep_operating_point.py`, `score_spliced_recall.py`, `build_spliced_recall_set.py`, `eval_test_split.py`, `measure_vandalism_rule.py`, `measure_object_scale.py`, `check_holdout_overlap.py`, `build_property_crime_manifest.py`, `optimize_weights.py`. Narrative and derivations in `docs/progress_report_violence_detection.md` §§27–32 and `docs/detection_performance_report.md`.